GAME DEVELOPMENT PORTFOLIO

チャ・スンジェ（Cha Seungjae）— ゲーム開発ポートフォリオ

Unity · Godot · GameMaker Studio
github.com/Chaseungjae ｜ jea3530@gmail.com

本ポートフォリオは、実際のGitHubリポジトリのコミット履歴とソースコードに基づき、
自分が開発した機能と修正したバグをコード単位でまとめた資料です。

作品一覧

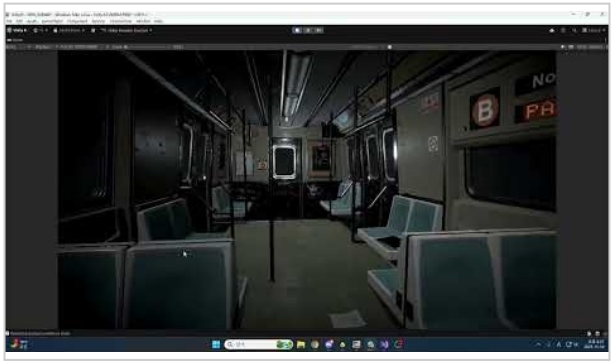
Overview

作品	ジャンル / エンジン	体制	主な担当	動画
AETHERIS 2026.04 – 2026.06	2Dヴァンサバライク・ローグライト Unity 6	5名	スキルシステム（10種以上）、スキルツリー、アイテム効果の適用・データ解析、倉庫、ボスパターン、ダンジョン演出	▶ 動画
God Is Death 2 2025.11 – 2025.12	ボスラッシュ・アクション GameMaker Studio (GML)	3名	ボスパターン／スキル、攻撃予告（テレグラフ）システム、ブラックホール重力場	▶ 動画
地下鉄ホラーゲーム 2025.10 – 2025.12	3Dホラー Unity (URP)	4名	一人称の移動／カメラ、モンスターAI、ジャンプスケア演出、照明演出、バグ修正	▶ 動画
2Dアクション プラットフォーマー 2025.05 – 2025.06	ホールドジャンプ・プラットフォーマー Godot 4 (GDScript)	個人	企画から実装まで全工程（チャージジャンプ、壁反射物理、ノックバック、演出）	▶ 動画（最終版） ＋開発進行動画5本（詳細ページ）

01 地下鉄ホラーゲーム

Unity 3D · チーム制作（4名）· 2025.10 – 2025.12

エンジン	Unity (URP), Cinemachine
体制	4名のチームプロジェクト（自分を含む）
担当	プレイヤー移動／カメラ、モンスターAIとジャンプスケア演出、バグ修正
リポジトリ	github.com/Chaseungjae/UnityProject
動画	youtu.be/jdeIvBlqOUw（プレイテスト動画）



▶ 地下鉄ホラーゲーム テストプレイ動画（YouTube）

地下鉄のホームと車内を舞台に、正常／異常な状況を見分けて次の電車へ進む3Dホラーゲームです。判断を誤るとモンスターが出現し、ジャンプスケア演出が発生します。

担当機能

- **一人称視点の移動システム** — Rigidbodyベースで、カメラ基準の方向ベクトルから移動を処理（PlayerMove.cs）
- **Cinemachineによるマウス視点操作** — 上下の視野角を制限し、感度をPlayerPrefsに保存（MyCustomCamera.cs）
- **モンスターAIとジャンプスケア演出** — NavMeshAgentによる巡回＋トリガー侵入時のカメラ強制カット・効果音・血のエフェクト（MonsterController.cs、202行を新規作成）
- **照明の明滅による恐怖演出** — コルーチンで5秒周期にライト強度を点滅（LightScript.cs、新規作成）
- **攻撃モーション中の位置固定ユーティリティ**（FixPositionDuringAttack.cs、新規作成）
- **ポストプロセッシングによる視覚効果** — 画面の雰囲気や恐怖感を高める演出、画面の明るさ設定、チュートリアル

コード例 — カメラ基準の移動ベクトル計算

```
// PlayerMove.cs
Vector3 forward = transform.forward; forward.y = 0f; forward.Normalize();
Vector3 right   = transform.right;   right.y   = 0f; right.Normalize();

// 方向ベクトルを合成した後に正規化 → 斜め移動で速度が上がるのを防止
Vector3 moveDir = (forward * inputDirection.y + right * inputDirection.x).normalized;
```

バグ修正事例

① **斜め移動の速度バグ** — 前後・左右の方向ベクトルを正規化せずに足すと、斜め移動時の速度が√2倍になる典型的なバグがありました。方向ベクトルの合成後に **.normalized** を適用し、どの方向でも同じ移動速度になるよう修正しました。

② **ステージ進行ロジックのバグ**（コミット 1aeaca4 「バグ修正」）— ステージの成功／失敗に関係なく、モンスターが常に同じ経路へ移動していました。コード上で原因を確認し、成功可否に応じて分岐するよう修正しました:

```
  if (gameManager.is_subway)
  {
-     StartDirectMove(secondTargetA);  // 成功/失敗に関係なく常に同じ経路
+     if (gameManager.is_success)
+     {
+         StartDirectMove(secondTargetB);
+         gameManager.stage_count += 1;
+     }
+     else
+     {
+         StartDirectMove(secondTargetA);
+         gameManager.stage_count = 0;
+     }
  }
```

同じコミットで、**next_stage()** 内の **is_subway** フラグのリセット漏れ（ステージ切替後に前の状態が残る問題）と、死亡後のリセット待機時間が10秒と長すぎた問題（5秒に短縮）も併せて修正しました。

③ **攻撃中にモンスターの位置がずれる（スライドする）バグ** — 攻撃アニメーションの再生中にモンスターの位置がずれる現象を確認し、攻撃開始時の位置を保存して、攻撃中は毎フレームその位置に固定するユーティリティスクリプトを新規作成して解決しました:

```
// FixPositionDuringAttack.cs
public void StartAttack() {
    isAttacking = true;
    fixedPosition = transform.position;   // 攻撃開始時の位置を保存
}
void Update() {
    if (isAttacking) transform.position = fixedPosition;   // 攻撃中は位置を固定
}
```

④ **壁抜けバグ**（コミット 33d5401 「壁抜けバグ修正」）— プレイヤーが壁をすり抜けてしまう問題を、地下鉄のプレハブおよびシーン設定（計23ファイル）を調整して修正しました。

02 2Dアクション・プラットフォーマー

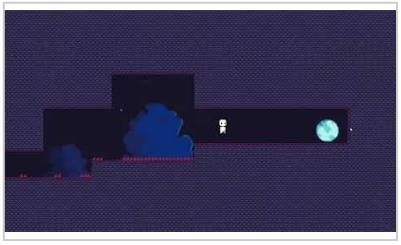
Godot 4 · GDScript · 個人制作 · 2025.05 – 2025.06

エンジン	Godot Engine (GDScript)
体制	個人制作（単独開発）
担当	企画から実装まで全工程（キャラクターコントローラー、物理、演出、サウンド）
リポジトリ	github.com/Chaseungjae/2Dproject
動画	youtu.be/_SWh5Mbt__I（最終版）／開発進行動画あり（下記）

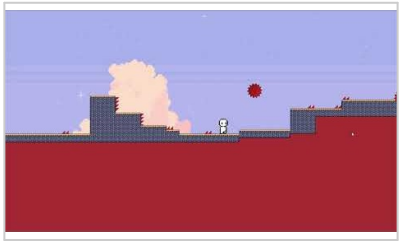


▶ godot テスト6（最終版・YouTube）

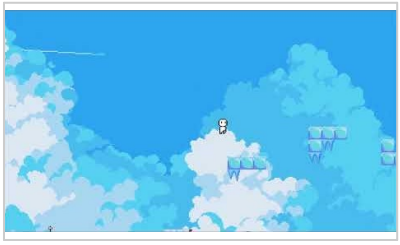
開発進行動画（YouTube）



▶ テスト3



▶ テスト4



▶ テスト5

その他：テスト1 ／ テスト2

ジャンプボタンを押し続けた時間でジャンプの高さと距離が決まる「ホールドジャンプ」と、壁に当たって跳ね返る「反射物理」を核にした、夜をテーマの2Dアクションプラットフォーマーです。危険な地形に触れると確率に応じて強さが変わるノックバックが発生し、ゴール到達時にクリアタイムを記録します。

担当機能（すべて自分で実装）

- **チャージジャンプ** — ボタンを押し続けた時間に応じて、ジャンプ力が **JUMP_FORCE** → **MAX_JUMP_FORCE** の範囲で線形補間され強くなる仕組み
- **壁反射（Wall Bounce）** — 衝突法線ベクトルから **bounce()** を計算し、ジャンプ力に比例して反射速度を決定
- **危険タイルのノックバック＆確率システム** — 危険タイルに触れるとカメラシェイク＋ノックバック。独自の確率変数（CTK）により、強い反応の確率が累積的に上昇
- **ジャンプ状態別のカメラオフセット切替** — 上昇／落下の状態に応じてカメラ位置を補間移動
- **クリアタイム記録とシーン遷移** — GameStateシングルトンに開始／終了時刻を記録し、経過時間を算出
- **レベルデザイン** — チャージジャンプと壁反射を活かしたステージ構成に注力

コード例 — チャージジャンプと壁反射

```
# character_body_2d.gd
# ジャンプボタンを押し続けた時間に比例してジャンプ力が増加
applied_jump_force = JUMP_FORCE + (MAX_JUMP_FORCE - JUMP_FORCE) * (jump_hold_time / MAX_HOLD_TIME)

# 壁に衝突したら入射角を基準に反射ベクトルを計算し、ジャンプ力に応じて減衰
var bounce_velocity = prev_velocity.bounce(normal)
bounce_velocity *= lerp(0.4, 1.0, jump_power_ratio)
bounce_velocity.x = clamp(bounce_velocity.x, -MAX_X_SPEED, MAX_X_SPEED)
```

03 ヴァンサバライク・ローグライト — AETHERIS

Unity 6 (2D), URP · チーム制作（5名）· 2026.04 – 2026.06

エンジン	Unity 6 (2D), URP, DOTween
体制	5名のチームプロジェクト（プログラマー3名・企画1名・アート1名。自分はプログラマー、コミット57件）
担当	戦闘スキルシステム、スキルツリー、アイテム効果の適用・データ解析、ボスパターン、ダンジョン演出、バランス調整、バグ修正
リポジトリ	github.com/ksw-0409/NEW_PROJECT
動画	youtu.be/38FY_WGQWFs（プレイ動画）



▶ AETHERIS プレイ動画（YouTube）

「AETHERIS」は、プログラマー3名・企画1名・アート1名の5名で制作した、2Dヴァンサバライク（ヴァンパイアサバイバーズ系）のローグライトです。リアルタイムで押し寄せるモンスターとボスを倒しながら進み、スキルツリーと装備システムでキャラクターを成長させます。

担当機能

- **戦闘スキルシステム** — 共通基底クラス（SkillBase）をもとに、ファイアボール・矢の雨・連鎖雷・メテオ・アイスレイン・スラム・スラッシュ・ソードウェーブ・回転斬り・シールドなど、10種以上のアクティブスキルを企画・実装し、バランスを調整
- **スキルツリー** — ノード式スキルツリーUIと、コスト／バランスの調整（SkillTreeUI.cs, SkillNode.cs）
- **装備・アイテムシステム** — 装備システムの企画・実装。アイテムデータのパース処理と、装備による効果・ステータスの適用（ItemDataRow, EquipmentManager）、倉庫（Stash）
- **ボスパターン設計** — VoidPriestBoss（誘導レーザー、封印の柱、ボイド隕石フィールド）、ボス撃破エフェクトとゴールドドロップ演出
- **ダンジョンの雰囲気演出** — 階層に応じて露出・彩度・ビネット・ブルームを補間適用するポストプロセス制御を新規作成（DungeonAtmosphereController.cs、144行）

コード例 — ポリモーフィズムによるスキル発動条件の設計

```
// SkillBase.cs - すべてのアクティブスキルの共通基底クラス
protected virtual bool RequiresEnemiesToCast() => true;   // 既定: 敵がいる時のみ発動

protected bool HasEnemiesToAttack() {
    var em = EnemyManager.Instance;
    if (em == null || em.activeEnemies == null) return true;
    for (int i = 0; i < em.activeEnemies.Count; i++) {
        var e = em.activeEnemies[i];
        if (e != null && e.gameObject.activeInHierarchy && !e.isDie) return true;
    }
    return false;
}

// ShieldSkill.cs - 防御系スキルは敵がいなくても発動すべきなので例外扱い
protected override bool RequiresEnemiesToCast() => false;
```

バグ修正事例

① ファイアボールが正しく発動しない問題（コミット 6817776 「ファイアボール解決」）— スキルデータの **projectilePrefab** の参照先が誤っていたため、正しいプレハブに差し替え、再発時に原因を追跡できるよう診断ログも追加しました。

② 敵を全滅させた後も攻撃スキルが発動し続ける問題（コミット bcd37d3）— 自動発動コルーチンがクールダウンごとに無条件でスキルを発動するため、周囲に敵がいなくても空振りしていました。基底クラスに仮想メソッドを追加し、攻撃系スキルは敵の存在を確認してから発動するよう修正しました:

```
   yield return new WaitForSeconds(GetCooldown());
-  Execute(this.transform);
+  if (!RequiresEnemiesToCast() || HasEnemiesToAttack())
+      Execute(this.transform);
```

③ ボスの誘導レーザーの判定バグ（コミット ceb6ac5 「バグ修正およびアイテム確率」）— VoidPriestBossの誘導レーザーが封印の柱とプレイヤーを同時に貫通した場合、「柱に当たっていない時のみ」プレイヤーにダメージを与える条件になっていたため、柱と一直線上に立つとレーザーに当たってもダメージを受けない判定バグがありました。ビームが表示される0.25秒の間、毎フレーム独立してプレイヤーとの衝突を再判定するようロジックを分離して解決しました。

④ パチンコ画面とのUI重なりバグ（コミット 000c349 「バグ修正」）— チームメンバーが実装したパチンコ（ベット）画面が開いている間も、プレイヤーのシールドエフェクトがUIの上に描画され続ける問題を、**OnEnable／OnDisable** でシールド表示のON/OFFを切り替えるよう修正しました。

04 ボスラッシュ・アクション — God Is Death 2

GameMaker Studio (GML) · チーム制作（3名）· 2025.11 – 2025.12

エンジン	GameMaker Studio (GML)
体制	3名のチームプロジェクト（自分を含む、コミット23件）
担当	ボスパターン／スキル実装、攻撃予告（テレグラフ）システム、バグ修正
リポジトリ	github.com/retser0319/GayMakerStudio
動画	youtu.be/cPjAI8rbP_c（プレイ動画）



▶ God Is Death 2 プレイ動画（YouTube）

「God Is Death 2」— 主人公ニーチェ（obj_Nietzsche）が、それぞれ異なるパターンを持つボスを順に撃破していく2Dボスラッシュ・アクションです。ボスを倒すたびにカード（強化）を1枚選んで成長する、ローグライト要素を備えています。

担当機能

- **攻撃予告（テレグラフ）システム** — obj_Danger_1〜8の8種の危険エリアオブジェクトを設計。一定時間の点滅警告のあとに実際の判定が発生する「予告→攻撃」の2段階構造を、共通パターンとして実装
- **ボス2〜4の多彩な攻撃パターン** — 雷（Lightning）、分身（Clone）、範囲攻撃など、ボスごとのスキルオブジェクトを多数作成
- **ブラックホール重力場ギミック** — 距離に反比例する引力でプレイヤーを引き寄せる物理演算を実装（obj_black_hole）
- **石像召喚ギミック** — stone_spawn／stone_statue オブジェクトで、フィールドに障害物兼攻撃判定を召喚
- **強化（カード）のバランス調整**
- **カメラ演出の調整、サウンド追加**

コード例 — ブラックホール重力場（距離に反比例する引力）

```
/// obj_black_hole / Step_0.gml - ブラックホールの引力 (hspd/vspd 加算方式)
var _dist = point_distance(x, y, obj_Nietzsche.x, obj_Nietzsche.y);

if (_dist < _max_range && _dist > _min_dist) {
    var _strength  = _max_power * (1 - (_dist / _max_range));  // 近いほど強く
    var _dir_angle  = point_direction(obj_Nietzsche.x, obj_Nietzsche.y, x, y);
    obj_Nietzsche.hspd += lengthdir_x(_strength, _dir_angle);   // プレイヤーの速度に直接力を加算
    obj_Nietzsche.vspd += lengthdir_y(_strength, _dir_angle);
}
```

バグ修正事例

① **ボスの撃破判定の不足**（コミット 0f3d4d3）— obj_Boss_2のHPが0以下になった時にオブジェクトを破棄する処理が不足していました。Stepイベントで **global.BossHP <= 0** を判定して **instance_destroy** を呼び、破棄時にカード報酬ルームへ遷移するDestroyイベントを追加しました:

```
    if(global.BossHP <= 400){
        global.Boss2Paze = 2
    }
+
+ if (global.BossHP <= 0) instance_destroy(self)    // 撃破判定を追加

    // Destroy イベントを新規追加
+ room_goto(Card_Room)    // 撃破後、カード（強化）報酬ルームへ遷移
```

② **フェーズ2でも攻撃間隔が変わらなかった問題** — ボスのHPが一定以下（フェーズ2）になっても、攻撃クールダウンがフェーズ1と同じ固定値（120〜240フレーム）のままで、後半の難易度が上がらない状態でした。フェーズ別のクールダウン変数を導入し、フェーズ2ではクールダウンを60フレームに短縮、使用可能な攻撃パターンも拡張しました:

```
- var pattern = choose(1,2,3,5)
+ global.attack_timer = 120
+ if(global.Boss2Paze == 2){
+     var pattern = choose(1,2,3,5)
+     global.attack_timer = 60      // フェーズ2: 攻撃間隔を半分に短縮
+ }
+ else var pattern = choose(1,2,3)
```

③ **条件式の演算子の統一** — 条件式に比較演算子（==）ではなく代入演算子（=）の記述（**if(global.Boss2Paze = 2)**）が残っていました。GMLでは動作する場合もありますが、意図が曖昧でバグの温床になるため == に統一しました。